

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 3 – FUNCTION CALLING & STRUCTURED OUTPUT TASK

Course Code:	CSA65	Course Title:	Generative AI and Large Language Models
CO Assessed:	CO2 – Precision (BL2, BL3)	Assessment:	Assessment Tool 3 – Function Calling & Structured Output Task
Weightage:	40% of CIA	Total Marks:	1 * 100 = 100 (1 Question1)
CO2 Statement:	<i>Apply prompt engineering techniques and utilize pre-trained Large Language Models through modern AI frameworks and APIs to solve real-world problems (BL2, BL3)</i>		
Student Name:		Reg. No.:	
Section / Batch:		Date:	

Instructions to Students

- Answer 1 question. Total: 100 Marks.
- Analyze the given scenario and identify the appropriate function(s), tool(s), parameters, and structured output required to solve the task.
- Design a valid function-calling schema with appropriate function names, parameters, data types, required fields, and descriptions based on the given requirements.
- Implement the function-calling workflow demonstrating the complete process: User Query → LLM → Function/Tool Call → Function Execution → Structured Response.
- Ensure that the generated output strictly follows the defined structured format or JSON schema and contains valid, relevant, and consistent information.
- Handle appropriate edge cases such as missing parameters, invalid inputs, ambiguous requests, incorrect data types, or unavailable information, and provide suitable responses without producing invalid function calls.
- Demonstrate the working implementation using suitable test cases, including normal inputs and at least one edge-case scenario.
- Clearly document the designed schema by explaining the purpose of each function, its parameters, data types, required fields, expected input, and expected output.
- Briefly justify the design decisions and explain how function calling and structured output improve the reliability, consistency, and usability of the LLM-based application.

Submission Requirements

Students should submit:

- **Source code / Jupyter Notebook**
- **Function-calling schema**
- **Structured output/JSON schema**
- **Execution screenshots**
- **Test cases and results**
- **Brief documentation explaining the schema and function-calling workflow**

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

SECTION A — FUNCTION CALLING & STRUCTURED OUTPUT TASK

1. Smart University Assistant

A university wants an AI assistant that can retrieve student attendance and examination information.

Task: Design two functions, `check_attendance()` and `get_exam_details()`. Define their parameters and structured JSON output. Demonstrate how the LLM identifies which function to call based on the student's query.

2. Online Food Ordering Assistant

An AI assistant should help customers search for food items and place orders.

Task: Design function schemas for `search_food()` and `place_order()`. Demonstrate function calling for a customer asking, “Find two vegetarian pizzas under ₹500 and place the order.”

3. Hospital Appointment Assistant

A hospital wants an AI assistant to help patients find doctors and schedule appointments.

Task: Design `find_doctor()` and `book_appointment()` functions. Show the complete user → LLM → function call → execution → structured response flow.

4. E-Commerce Product Assistant

An online store wants an AI assistant that can search products based on category, price, and brand.

Task: Design a `search_products()` function with appropriate parameters and structured output. Demonstrate how the assistant handles a query with missing product information.

5. Banking Assistant

A bank wants an AI assistant that can retrieve account balance and recent transactions.

Task: Design `get_account_balance()` and `get_transactions()` functions. Ensure that the output follows a predefined JSON schema and demonstrate appropriate handling of invalid account information.

6. Travel Booking Assistant

A travel company wants an AI assistant that can search flights and hotels.

Task: Design function schemas for `search_flights()` and `search_hotels()`. Demonstrate how the assistant selects the appropriate function for different user requests.

7. Employee Leave Management

An organization wants an AI assistant that can check employee leave balance and apply for leave.

Task: Design `get_leave_balance()` and `apply_leave()` functions. Demonstrate structured output and handling of missing dates or leave type.

8. Library Management Assistant

A college library wants an AI assistant to search books and check book availability.

Task: Design `search_book()` and `check_availability()` functions. Demonstrate function calling using at least three different user queries.

9. Customer Support Ticket System

A company wants an AI assistant that can create and track customer support tickets.

Task: Design `create_ticket()` and `track_ticket()` functions. Define suitable parameters, required fields, and structured responses.

10. Weather Information Assistant

An application wants an AI assistant that retrieves weather information for a specified location.

Task: Design a `get_weather()` function. Demonstrate how the assistant handles queries with complete, incomplete, and ambiguous locations.

11. Structured Resume Analyzer

A recruitment company wants an LLM to analyze resumes and return structured candidate information.

Task: Design a structured output schema containing candidate name, skills, education, experience, and suitability score. Explain how structured prompting improves consistency.

12. Student Feedback Analyzer

A university receives large amounts of student feedback.

Task: Design a structured prompt that classifies feedback into categories such as Academic, Infrastructure, Faculty, Examination, and Other, and returns sentiment and priority in JSON format.

13. Invoice Information Extractor

A company wants to automatically extract information from invoices.

Task: Design a structured output schema containing invoice number, date, vendor, total amount, tax, and payment status. Demonstrate how the LLM should respond when a field is missing.

14. Legal Document Analyzer

A legal firm wants an AI system to extract important clauses from contracts.

Task: Design a structured prompt that extracts clause type, clause description, risk level, and recommended action. Explain how constraints improve output reliability.

15. Medical Report Summarizer

A hospital wants to summarize medical reports into a fixed structure.

Task: Design a structured output containing patient summary, findings, observations, and recommended follow-up. Explain how the prompt should prevent the model from inventing information.

16. Product Review Classification

An e-commerce company wants to classify customer reviews.

Task: Design a structured prompt that returns sentiment, product category, major complaint, and recommendation. Use suitable few-shot examples to improve classification consistency.

17. Zero-Shot vs Few-Shot Classification

A company wants to classify customer queries into Billing, Technical Support, Account, and General.

Task: Design both a zero-shot and few-shot prompt for the same task. Compare their expected performance and explain when few-shot prompting would be preferable.

18. One-Shot Email Classification

A company wants to classify incoming emails as Urgent or Non-Urgent.

Task: Design a one-shot prompt containing one example and produce a structured JSON response. Explain why one-shot prompting is suitable for this scenario.

19. Few-Shot Text Formatting

A company wants an LLM to convert informal customer messages into professional support requests.

Task: Design a few-shot prompt with at least three examples and require the output to follow a fixed structure containing issue, priority, and requested_action.

20. Chain-of-Thought Reasoning Task

A financial application needs an LLM to analyze a multi-step calculation problem.

Task: Design an appropriate prompt that encourages reliable multi-step reasoning while returning only the required final structured result. Explain why a reasoning-oriented prompting strategy is appropriate.

21. Prompt Optimization for Customer Support

A poorly designed customer-support prompt produces lengthy and inconsistent answers.

Task: Evaluate the given scenario and redesign the prompt using clear instructions, context, constraints, and output formatting. Explain how your revised prompt improves quality.

22. Prompt Evaluation Using Test Cases

An organization has developed a prompt for summarizing customer complaints, but the output varies significantly.

Task: Create at least five test cases and evaluate the prompt based on consistency, relevance, completeness, and adherence to the expected format. Refine the prompt based on your evaluation.

23. LLM API-Based Chatbot

A company wants to develop a chatbot using an LLM API such as OpenAI, Gemini, or Hugging Face.

Task: Design a simple API-based chatbot and define the system prompt, user input format, expected response structure, and error-handling mechanism.

24. API-Based Structured Information Extraction

A company wants to use an LLM API to extract customer information from emails.

Task: Design the prompt and structured output schema. Demonstrate how the API response can be converted into a predictable JSON object.

25. Multi-Function Customer Service Agent

A customer-service assistant should be able to check order status, cancel an order, and create a complaint.

Task: Design three function schemas and explain how the LLM determines which function should be called for different customer requests.

26. Sequential Function Calling

A travel assistant receives the request:

“Find a hotel in Chennai for two nights and calculate the total estimated cost.”

Task: Design a workflow in which the LLM calls multiple functions sequentially, such as `search_hotels()` followed by `calculate_cost()`. Show the structured responses at each stage.

27. Function Calling with Missing Parameters

An employee asks:

“Apply leave for next week.”

The leave system requires leave type, start date, end date, and employee ID.

Task: Design the function schema and explain how the LLM should identify missing parameters before calling the function.

28. Function Calling with Ambiguous Input

A customer says:

“Cancel my order.”

The customer has multiple active orders.

Task: Design an appropriate function schema and workflow for handling ambiguity without accidentally cancelling the wrong order.

29. Function Calling with Validation

An online shopping assistant receives a request to purchase 100 units of a product, but the product is available only in a quantity of 20.

Task: Design a function-calling workflow that validates availability before completing the order. Define suitable structured outputs for success and failure.

30. Secure Function Calling

A banking assistant can check account details and initiate money transfers.

Task: Design a secure function-calling workflow that requires confirmation before executing a transfer. Identify the parameters, validation rules, and structured response required.

31. Transfer Learning for Domain Adaptation

A general-purpose LLM performs poorly when answering questions about a company's internal technical documentation.

Task: Explain how transfer learning can be used to adapt the model to the company's domain. Propose an appropriate fine-tuning or prompting strategy and explain your choice.

32. LoRA-Based Fine-Tuning

A healthcare organization wants to adapt an existing LLM to generate standardized medical summaries but has limited computational resources.

Task: Explain how LoRA/PEFT could be used instead of full fine-tuning. Describe the expected workflow and advantages of the approach.

33. PEFT Strategy Selection

A company wants to customize a large LLM but cannot afford to update all model parameters.

Task: Compare full fine-tuning with PEFT/LoRA and recommend an appropriate approach. Explain how the selected approach reduces computational and memory requirements.

34. Domain-Specific Prompting vs Fine-Tuning

A legal company wants its LLM to consistently produce responses using legal terminology and a specific document format.

Task: Determine whether prompt engineering, few-shot prompting, fine-tuning, or PEFT would be most appropriate. Justify your decision based on the scenario.

35. LLM Architecture and Function Calling

A developer wants to understand how an LLM processes a request before generating a function call.

Task: Explain the role of tokenization, embeddings, transformer layers, attention mechanisms, and output generation in the function-calling process. Represent the process using a suitable architecture diagram.

36. Pre-Training vs Fine-Tuning

A company wants to build a specialized customer-service LLM.

Task: Explain what knowledge the model would acquire during pre-training and what would be adapted during fine-tuning. Design a suitable workflow for adapting a pre-trained model to customer-service tasks.

37. API Comparison Task

A development team is deciding between OpenAI, Gemini, and Hugging Face for an LLM application.

Task: Compare the three approaches based on API accessibility, model availability, customization, deployment considerations, structured output/function-calling support, and ease of integration. Recommend a suitable option for the given application.

38. Prompt Best-Practice Evaluation

An AI assistant uses the following prompt:

“Tell me everything about this product and give the customer a good answer.”

Task: Identify the weaknesses in the prompt and redesign it using role, context, task instructions, constraints, and a specified output format. Explain how each modification improves the prompt.

39. End-to-End AI Assistant

A university wants an AI assistant that can answer academic questions, retrieve student information, calculate attendance, and generate emails.

Task: Design an end-to-end solution using an LLM API, structured prompting, function calling, and structured output. Define at least three functions and demonstrate how the assistant handles different types of requests.

40. Integrated Enterprise LLM Application

An enterprise wants to build an AI assistant that uses a pre-trained LLM, domain adaptation, prompt engineering, function calling, and structured outputs.

Task: Design an end-to-end architecture for the system. Your solution should show:

- LLM architecture
- Pre-training/fine-tuning or PEFT strategy
- Prompt design
- Function/tool definitions
- Structured output schema
- API integration
- Function-calling workflow
- Error and edge-case handling
- Prompt evaluation and refinement

Also justify the technologies and techniques selected for the proposed system.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Schema Design	30%	Correctly designs a function-calling schema and structured (JSON) output format	Mostly correct schema with minor issues	Schema has significant errors	Schema is fundamentally incorrect or missing
Output Validity	25%	LLM reliably produces valid structured output matching the schema	Mostly valid output with occasional errors	Output is frequently invalid or inconsistent	Output rarely or never matches the schema

Edge-Case Handling	20%	Handles edge cases (missing fields, ambiguous input) gracefully	Handles common cases well	Handles only the simplest cases	Fails on basic cases
Function-Call Flow Demo	15%	Clear demo of the function-call to structured-response flow	Demo covers the main flow	Demo is incomplete	No functional demo presented
Schema Documentation	10%	Well-documented schema design rationale	Adequate documentation	Minimal documentation	No documentation provided

Marks Evaluation:

Question No.	Schema Design(30%)	Output Validity (25%)	Edge-Case Handling (20%)	Function-Call Flow Demo(15%)	Schema Documentation (10%)	Total Marks (100)
Q1						
Overall Total (100)						

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____